

Gattini Cafe

API REST con Django e autenticazione JWT

Una caffetteria felina ha bisogno del vostro aiuto!

Lo Scenario

Benvenuti nel meraviglioso mondo di Gattini Cafe, la prima caffetteria al mondo interamente gestita (e frequentata) da gatti esigenti con un palato raffinato!

Pallino, il proprietario - un maestoso Maine Coon con un debole per i cappuccini - ha un problema: il suo sistema di ordinazioni è ancora cartaceo, e i suoi colleghi felini continuano a perdere i foglietti tra un pisolino e l'altro. Ha assunto voi, sviluppatori di fiducia, per costruire finalmente un'API REST moderna che gestisca il menu e gli ordini del suo localino.

Il database è già pronto (ve lo forniamo noi, non dovete preoccuparvi della struttura iniziale). Il vostro compito è sviluppare il backend con Django e Django REST Framework, proteggendo le risorse sensibili con autenticazione JWT.

Nota sul database

Trovate il file gattini_cafe.db nella cartella di consegna dell'esercizio.

Configurate Django per usare questo SQLite esistente come database del progetto.

Il database contiene già dati di esempio: categorie, prodotti, ordini e utenti di test.

Le password degli utenti di test sono state resettate -> usate il comando `createsuperuser` per accedere come admin.

Struttura del Database

Il database contiene le seguenti tabelle principali:

Tabella: categoria

- id - chiave primaria
- nome - nome della categoria (es. 'Bevande Calde')
- descrizione - testo descrittivo facoltativo

Tabella: prodotto

- id, nome, descrizione
- prezzo - prezzo in euro (float)
- disponibile - 1 se disponibile, 0 se esaurito
- categoria_id - FK verso categoria
- immagine_url - URL immagine (facoltativo)

Tabella: ordine

- id, utente_id (FK verso auth_user di Django)
- data_ordine - timestamp dell'ordine
- stato - valori: 'in_attesa', 'in_preparazione', 'completato', 'annullato'
- totale - importo totale calcolato
- note - istruzioni speciali (es. 'senza lattosio')

Tabella: ordine_prodotto (tabella ponte)

- ordine_id + prodotto_id - FK verso ordine e prodotto
- quantita - numero di unita ordinate

Requisiti dell'Applicazione

1. Setup e Configurazione

Configurate un progetto Django con le seguenti dipendenze:

```
djangoframework
djangoframework-simplejwt
django-cors-headers # opzionale ma consigliato
```

Configurate settings.py per usare il file gattini_cafe.db fornito:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'gattini_cafe.db',
    }
}
```

2. Modelli Django

Definite i modelli corrispondenti alle tabelle del database. Attenzione: la tabella auth_user è già gestita da Django tramite il modello User built-in. Per le altre tabelle dovrete creare modelli con managed = False oppure eseguire le migrazioni su un db fresco (a vostra scelta).

3. Autenticazione JWT

Implementate l'autenticazione tramite token JWT usando Simple JWT:

Endpoint di autenticazione da implementare:

Metodo	Endpoint	Auth JWT	Descrizione
POST	/api/auth/register/	No	Registrazione nuovo utente
POST	/api/auth/login/	No	Login: restituisce access + refresh token
POST	/api/auth/token/refresh/	No	Rinnova l'access token
GET	/api/auth/me/	Sì	Dati dell'utente autenticato

Il token JWT va incluso nelle richieste protette nell'header Authorization:

Authorization: Bearer <access_token>

4. Endpoint del Menu (Pubblici)

Questi endpoint sono accessibili da chiunque, anche senza autenticazione:

Metodo	Endpoint	Auth JWT	Descrizione
GET	/api/categorie/	No	Lista di tutte le categorie
GET	/api/categorie/{id}/	No	Dettaglio singola categoria
GET	/api/prodotti/	No	Lista di tutti i prodotti
GET	/api/prodotti/{id}/	No	Dettaglio singolo prodotto

La lista prodotti deve supportare i seguenti query parameter **facoltativi**:

- ?categoria=<id> - filtra per categoria
- ?disponibile=true - mostra solo prodotti disponibili
- ?search=<testo> - ricerca per nome o descrizione

5. Endpoint di Gestione (Solo Admin)

Questi endpoint richiedono autenticazione JWT e che l'utente abbia is_staff=True:

Metodo	Endpoint	Auth JWT	Descrizione
POST	/api/prodotti/	Sì	Crea nuovo prodotto
PUT	/api/prodotti/{id}/	Sì	Modifica prodotto
PATCH	/api/prodotti/{id}/	Sì	Aggiornamento parziale prodotto
DELETE	/api/prodotti/{id}/	Sì	Elimina prodotto
POST	/api/categorie/	Sì	Crea nuova categoria
PUT	/api/categorie/{id}/	Sì	Modifica categoria
DELETE	/api/categorie/{id}/	Sì	Elimina categoria

6. Endpoint Ordini (Utente Autenticato)

Ogni utente autenticato può creare e visualizzare solo i propri ordini. Gli admin possono vedere tutti gli ordini.

Metodo	Endpoint	Auth JWT	Descrizione
GET	/api/ordini/	Si	Lista ordini (propri o tutti se admin)
POST	/api/ordini/	Si	Crea nuovo ordine
GET	/api/ordini/{id}/	Si	Dettaglio ordine
PATCH	/api/ordini/{id}/stato/	Si	Aggiorna stato (solo admin)

Il payload per creare un ordine deve avere questa struttura:

```
{
  "note": "Senza glutine!",
  "prodotti": [
    {"prodotto_id": 1, "quantita": 2},
    {"prodotto_id": 3, "quantita": 1}
  ]
}
```

Il campo totale deve essere calcolato automaticamente dal server sommando prezzo * quantita per ogni prodotto dell'ordine.

Requisiti Tecnici e Valutazione

Struttura del Progetto Consigliata

```
gattini_cafe_project/
├── manage.py
├── gattini_cafe.db      <- il file che vi forniamo
├── requirements.txt
├── config/
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── api/
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── urls.py
│   └── permissions.py
```

Criteri di Valutazione

Criterio	Punti
Setup progetto, configurazione Django e DB	10
Modelli e serializers corretti	15
Endpoint pubblici del menu (GET)	15
Autenticazione JWT (registrazione, login, refresh, me)	20
Permessi corretti (pubblico / utente / admin)	15
Endpoint ordini con calcolo totale automatico	20
Qualità del codice, struttura e README	5

Funzionalità Bonus (Facoltative)

Le seguenti funzionalità non sono obbligatorie ma portano punti extra:

1. Paginazione: aggiungete paginazione alla lista prodotti e alla lista ordini
2. Endpoint statistiche admin: GET /api/admin/stats/ con numero ordini per stato, prodotto più venduto, incasso totale del giorno
3. Upload immagine prodotto: permettete all'admin di caricare un'immagine per il prodotto
4. Filtro ordini per data: ?data_da=2024-11-01&data_a=2024-11-30
5. Test unitari: scrivete almeno 5 test con pytest o il test runner di Django

Bonus Extra: Client per le API (+10 punti)

Come sfida aggiuntiva, potete sviluppare un client che consumi le vostre API. Il linguaggio è completamente libero: JavaScript, Python, Java, C#, C++, Kotlin, Swift... scegliete quello con cui vi sentite più a vostro agio.

Il client deve essere incluso nello stesso repository GitHub in una cartella separata chiamata client/ e deve implementare almeno:

- Login con salvataggio del token JWT per le richieste successive
- Visualizzazione del menu: categorie e prodotti disponibili
- Creazione di un ordine con uno o più prodotti

Non viene valutata l'estetica ma la correttezza delle chiamate HTTP, la gestione del token nell'header Authorization e la gestione degli errori (es. token scaduto, prodotto non disponibile).

Suggerimento
Se usate JavaScript/browser ricordatevi di abilitare CORS nel backend Django.
Se usate un linguaggio compilato (Java, C#, C++) va bene anche una semplice app a riga di comando.
Documentate nel README come avviare anche il client.

Modalità di Consegna

File da consegnare
La consegna avviene tramite repository GitHub pubblico.
Il repository deve contenere tutto il codice del progetto Django.
Includete requirements.txt, README.md e un .gitignore per Python/Django.
Se avete sviluppato il client, includetelo nella cartella client/.

Il README deve contenere almeno:

- Istruzioni di installazione (virtualenv, pip install, migrazioni)
- Come avviare il server di sviluppo
- Credenziali di un utente admin per i test
- Almeno 3 esempi di chiamate API con curl o httpie
- (Se presente il client) istruzioni per avviare anche il client

Buon lavoro! Pallino il Maine Coon vi ringrazia da parte di tutto il Gattini Cafe.